

# Change Management Field Manual

Memorize the whole deck the easy way: one story, the exact exam points, and a memory hook on every topic.

Module **MMC-01a**    Lecturer **Prof. Dr. M. Górka**    Source **4.pptx** · **135 slides**

**THD** · **NuW** · **Mechatronic & CPS**

## 🔄 Meet Ada — your memory anchor

Ada is the keeper of an electric car called “Volt.” Its blueprint was frozen long ago (PRD freeze), but the real world keeps poking it: a new law, a failed test, a supplier swaps a plug. Every slide just answers one question: “How do we let the product change SAFELY — as few changes as possible, but as many as needed — and log every version perfectly?” Keep Ada and her ever-changing car in your head and the chapter tells itself.

★ **MASTER SPIRIT** → “As few changes as possible, but as many as needed.” (Risk Mgmt stopped things going wrong; Change Mgmt manages what must change anyway.)

# See the skeleton before the bones

Change Management is **step 4** of System Engineering, living between PRD freeze and Re-lease:

1 Basics → 2 Requirements → 3 Risk → **4 CHANGE ★** → 5 Ideation → (6 Design) → (7 Test) → (8 Project Setup)

The chapter splits into a few big blocks. Learn this shape and everything else bolts onto one of these boxes:



**Fundamentals** (what/when/where/why) → **Version & Document control** → **Impact & Requirement changes** → **Product Life Cycle + BCG** → **Configuration & Variant management.**

---

# The Memory Movie

Don't memorize sentences — your brain hates random letters. Instead **close your eyes and play this 8-scene movie** of Volt the electric car's whole life, in slide order. Each picture is the point, so when you replay the movie, the exam answer just appears. Read it 3 times, then try to replay it with your eyes shut.



## SCENE 1

A **colour-changing chameleon** sits on Volt. Changing by nature = not CM; re-painting the actual blueprint = CM. → **What change management IS.**



## SCENE 2

Four nested **Russian dolls** tip out: a whole fleet → a model-year → a HW+SW combo → one tiny circuit board. → **VM ▶ LCM ▶ Config-CM ▶ Change-CM (objects grow as dolls shrink).**



## SCENE 3

The frozen **blueprint cracks** — one crack labelled “new LAW,” another “failed TEST.” → **PRD freeze is soft; “have-to” changes break in.**



## SCENE 4

Ada opens a **logbook** stamped 3.9.4 and **never erases** — she only adds a new page. → **Version & document control.**



## SCENE 5

A **snowball** rolls UP the hill, getting smaller: the later the root cause, the cheaper the fix. → **Inversed Rule of Ten.**



## SCENE 6

Volt rides a **roller-coaster**: pay to climb → peak & earn → roll down & get scrapped. → **Product life-cycle curve (phases 0-5).**



#### SCENE 7

A **farmyard** appears: a Star, a Cash Cow, a Question Mark, a Poor Dog. → **BCG matrix**.



#### SCENE 8

Ada snaps **LEGO bricks** into one kit and lines up 3 **paint-swatch** versions of the car. → **Configuration management + Variant management**.



Replay in one breath: **chameleon** → **dolls** → **cracked blueprint** → **never-erase logbook** → **uphill snowball** → **roller-coaster** → **farmyard** → **LEGO kit + paint swatches**. That's the whole chapter — no letters needed.



## The farmyard (BCG quadrants)

Picture the four animals on a field of **market growth (up)** × **your-share-vs-rival (right)**:



### Star

High growth, high share, best product. **Invest or skim without losing share**.



### Cash Cow

Low growth, high share. **Milk it — no discounts**.



### Question Mark

High-growth market, low share, new product. **Invest from elsewhere or discontinue**.



### Poor Dog

Low growth, low share. **Terminate (latest when margin/DB < 0)**.



## Picture-hooks for the tricky lists

Same trick for every list that wouldn't stick — turn it into one strong, weird image:



### The 4 cousins

Russian dolls, biggest first: Variant (fleet) ▶ Lifecycle (model-year) ▶ Config (HW+SW combo) ▶ Change (one PCB). **More objects as the doll shrinks.**



### Effort grows with 4

A weightlifter under 4 plates: Complexity · late Detection · Overlapping changes · Variants. **“Tangled, Late, Crowded, Many.”**



### 4 phases of WHEN

A clock chimes 4×: before SOP (most changes) → launch (critical) → SOP–EOP (fix & cost-down) → after EOP (learn). **Build·Launch·Fix·Learn.**



### WHERE = 2 linked drawers

A project drawer (whole change) chained to a PDM drawer (single object). **Unchained = branched data.**



### 3 TYPES of change

A balloon you inflate (enlarge), pop a piece off (reduce), or squeeze into a new shape (change). **Inflate · Pop · Reshape.**



### SIZE of change

A tortoise tiptoes (better phone = evolution) vs a rocket leaps to a new island (smartphone = disruption). **Tortoise vs rocket.**



### Version 3.9.4

Three dials: big MAJOR (re-design), middle MINOR (feature — rolls past 9 to 3.10.4!), small MICRO (patch). **Never overwrite.**



### Inversed Rule of Ten

A snowball rolling UP the hill shrinks: the later the root cause, the cheaper the fix. **(Inverse of the normal snowball.)**



### Config 5-step

O-I-C-S-V: Organize → Identify → Control (change lives here!) → Status-accounting → Verify/audit.



### Variant decision

A see-saw of base BC vs derivation BC → read the SUM: positive = implement, negative = stop/optimize. **Case 3 is the trap.**



**How to actually lock it in:** (1) read a scene, (2) **shut your eyes and re-see the picture**, (3) say the real meaning out loud. Do the 8-scene movie + picture-hooks twice today and once tomorrow — then test yourself with the **flashcards**. Seeing beats reading; testing beats re-reading.

---

# Taught like a story — with the points you must write

## 01 What *is* change management? (and what it is NOT)

### STORY

A **chameleon** sits on Volt's hood, changing colour — but it was built to do that, so that's **not** change management. Re-drawing the car's actual blueprint *is*. Careful: the word also means the *organizational/people* kind (behaviours, strategy, structures) — but **in this course we ignore that**. We mean **engineering** change management = changing a **product** or its **parts**.

### WRITE IN EXAM — DEFINITION

- **Engineering change management** (Ger. Änderungsmanagement, ÄM) = the **process of changing a product or parts of it**.
- The *other* CM (Ger. Veränderungsmanagement) = **organizational** change (processes, behaviours, strategy, systems, structures) → **not** our topic.
- **Content**: changes of requests & requirements must be **aligned, tracked, followed up**.
- **Target**: **as few changes as possible** (stable development) **but as many as needed** (sellable product).

 Chameleon changes **on purpose** = no CM. Re-paint the **blueprint** = CM. Goal = “**few but enough**.”

## 02 Four cousins, all about “what object?”

### STORY

Tip out **Russian dolls**, biggest to smallest: the whole **fleet of variants** (VM), a **model-year generation** E90→F30→G20 (LCM), a **HW+SW combo** (Config-CM), and the tiniest doll, **one circuit board** (Change-CM). The smaller the doll, the **MORE** of them there are.

 **WRITE IN EXAM — OBJECT INCREASES TOP→DOWN**

- **Variant management (VM):** products/variants — e.g. sedan, convertible.
- **Product life cycle management (LCM):** new generations — e.g. E90→F30→G20.
- **Configuration management (CM):** configurations = HW+SW combos — e.g. ECU SW 3.7.2 + d-sample PCB.
- **Change management (CM):** components — e.g. d-sample PCB.



Dolls shrink  $V \rightarrow L \rightarrow C \rightarrow C$ ; counts grow. Two “CM”s — Configuration vs Change — don't mix!

### 03 The 5 W's of change (Effort · When · Where · For whom · Why)

 **STORY**

Ada interrogates every change like a journalist: how *hard*, *when*, *where*, *for whom*, *why*.

 **WRITE IN EXAM**

- **Effort** grows with: **system complexity · late point of detection (Rule of Ten) · overlapping (parallel) changes · variants.**
- **When — 4 phases:** Until **SOP** (most changes) → **Launch management** (just before SOP, critical) → **SOP→EOP** (trouble-shooting + cost-down) → **After EOP** (record field issues, improve next product).
- **Where — 2 linked locations:** (1) **project-management level** (whole change) + (2) **where the original object was made** (e.g. PDM → drawing/BoM). **Both MUST be linked**, else the data branches.
- **For whom:** all **internal** AND **bought/supplied** products; track suppliers, nominate only **ISO 9001-certified** ones, put the reporting duty **in the contract**.
- **Why:** design-for-upgrade capability · traceability (versioning) · releasable/conform product · cost control · avoid extra work/missed deadlines/dev issues · high quality.



Effort = “Tangled, Late, Crowded, Many.” When = **Build·Launch·Fix·Learn**. Where = **two drawers, one chain**.



## 04 Level, Type & Size of a change

### STORY

Sort every change three ways — how *high-level*, what *kind*, how *big a leap*. A balloon you can inflate, pop, or reshape; a tortoise step or a rocket jump.

### WRITE IN EXAM

- **Level:** the higher the level of change, the **fewer** there are.
- **3 Types:** **Scope-Enlargement** (add new content) · **Scope-Reduction** (cancel content) · **Scope-Change** (change existing content).
- **2 Sizes:** **Evolutionary** = gradual improvement (better phone; risk = long-term failure if never disruptive). **Revolutionary/Disruptive** = radical rethink (smartphone vs telephone; risk = never establishes).



Type = Inflate · Pop · Reshape. Size = Tortoise tiptoe vs rocket leap.

## 05 Version control & Document control (the logbook)

### STORY

Ada's golden rule: **never erase** — **always add a new version**. Her logbook shows the object's whole history so anyone can roll back. Each object is named by **ID + version** (requirement #073 isn't enough — v1.0 or v1.1?).

#### WRITE IN EXAM — VERSION CONTROL

- **Motivation:** traceability over the whole life cycle (change frequency & object evolution visible; roll-back possible).
- **No overwriting** → create a **new version**; version number is an **attribute** (auto-counter in pro tools).
- **Identify** = ID/name + **version number** together.
- **Schemes:** single-level numeric (0,1,2...) · alphabetical (a,b,...,aa,ab...) · **two-stage** (a.0...b.0 or 1.1,1.2,2.0).
- **Semantic MAJOR.MINOR.MICRO:** Major = re-design (before the dot) · Minor = feature (after the dot) · Micro = patch. From 3.9.4 → 4.9.4 (major) / 3.10.4 (minor — **not** 4.x!) / 3.9.5 (micro).
- **Process:** like “new requirements” — change into pool → *acceptable?* → no: decline w/ reason (or modify & re-loop) → yes: **release new version**.
- **Content to log:** object · status · version no. · **What · Why · For-whom · Who · When · Who-approved · When-approved.**

#### WRITE IN EXAM — DOCUMENT CONTROL (ISO 9001)

- Information must be **available where/when needed** · **protected (confidentiality, integrity)** · **managed for distribution/access/discovery/use** · **stored with preserved readability** · **change-monitored** · **external docs marked.**
- *Available where needed* → safety manuals **at the machine**, not just on a drive.
- *Readability for 30 years* → can you still read that CD/USB and its **file format** later?



Version dials = **big·middle(rolls past 9!)·small**. Content = **detective's W-form**. Docs = **right place · protected · readable in 30 yrs · external = stamped.**

## 06 Impact of changes (+ the INVERSED Rule of Ten)

#### STORY

One little change ripples everywhere. Example: you started the endurance test with the **old round plug**, now everything switches to **rectangular plugs** — stop & redo? swap mid-test? ignore it?

#### WRITE IN EXAM — IMPACT

- Increased complexity (variants/configs) → quality issues & unhappy customers · time delay · worse cost/benefit · **breaks the PRD** · worst case **the whole project is stopped**.

#### WRITE IN EXAM — INVERSED RULE OF TEN

- The **later the root cause** (NOT the later the detection!) sits in the design process, the **cheaper & easier** to change.
- Cheap (late root cause) → expensive (early root cause): **update design → update assembly process → change concept → redesign the test → change the whole system**.

 A **snowball rolling UP** the hill shrinks — late root cause = small fix. (The inverse of the normal  snowball.)

## 07 Design, Concept & Requirement changes

#### STORY

The frozen **blueprint cracks**. Two crack-types: ones you have *no choice* about, and ones you *choose*.

#### WRITE IN EXAM

- **Design & concept change — caused by:** failed tests · realization issues (HW won't work / too costly → switch to SW) · needing a **re-design or new concept**.
- **PRD freeze:** ideal = hard (no changes after); reality = **soft** → changes happen.
- **“Have-to” changes** (no choice, before launch): a **new law** effective before launch · a **failed verification** → relax an **over-engineered** requirement (else change design/concept).
- **“Can-be” changes** (your call): new/extended **applications** · **company strategy** · **customer expectation** (B2B = extra cost) · **tool kits** (round→rectangular plugs) · **cultural/political** (no combustion engines).
- **How to handle:** re-open requirement process → **impact analysis** (resources + risks) → create a **CR** → **CCB**/project lead decides & records → **prioritize** (cost/benefit) → **plan**. Valid config = **base line + all CRs**; then update → document → implement → **track verification** → release.



Cracks = “**have-to**” (law / failed test) vs “**can-be**” (your call). Path: **CR** → **CCB** → **prioritize** → **plan**.

## 08 Product life cycle (the roller-coaster)

### STORY

Volt rides a **roller-coaster**: you **pay to climb**, you **peak and earn**, then you **roll down and get scrapped**. Run the whole ride again = a **new generation** (managed by PLM/LCM).

### WRITE IN EXAM

- **Repeat reasons**: *Positive* — new customer requirements, intrinsic advancement, state-of-the-art advances. *Negative* — competing products, quality issues, bad reviews, cost reduction.
- **Networking**: **Horizontal** = across generations (usually forward-only). **Vertical** = across products, **bidirectional**, strength = “technical proximity” (cars→trucks > cars→bicycles).
- **Branches (not linear)**: continue · **merge** · **split** · **die** (e.g. Linux distribution timeline).
- **Cycle length**: trend = shorter (fast fashion) vs sustainability. Depends on **industry** (auto ~7 yrs vs phone ~1 yr), **new technology**, and **starting point**.
- **Starting point / derivation**: **New development** (nothing reused) · **Wide derivation** (basic features inherited) · **Close derivation** (most carried over). E.g. landline → (wide: SIM) → mobile → (close: bigger display) → smartphone.



Roller-coaster phases below. Horizontal = generations (forward); Vertical = products (both ways).

## 09 BCG Matrix (the farmyard)

### STORY

A farmyard of products: a shooting **Star**, a **Cash Cow** to milk, a **Question Mark** new arrival, a sad **Poor Dog**. Boston Consulting Group plots them on **market growth** × **relative market share**. It only records the **actual** situation — it does **not** forecast.

#### WRITE IN EXAM

-  **Stars** — high growth, high share, best product → **invest** (own cash flow) or skim without losing share.
-  **Cash Cow** — low growth, high share → “**milk**” (skim), **no discounts**.
-  **Question Marks** — high-growth, low share, new product → **invest from other sources or discontinue**.
-  **Poor Dog** — low growth, low share, discontinued → **terminate** (latest when contribution margin / DB < 0).
- **Typical order:**  →  →  → . Question-marks make future stars; cash cows finance; products support each other.
- **Relative market share** = company share ÷ competitor share (>1 = you lead).



Farmyard: **Star** → **Cow** → **Question** → **Dog**. Growth ↑ × your-share-vs-rival.

## 10 Configuration management (snap the LEGO together)

#### STORY

Ada snaps the chosen **LEGO bricks** (one inverter version + one motor version + one damping version) into a single **working combo** and writes down exactly which bricks. **Configuration** management cares about the **combo's properties**; **change** management cares about the **single brick**. (Same “CM” — different job!) It runs even after SOP, owned by a **configuration manager**.

#### WRITE IN EXAM

- **Targets:** handle variant complexity · manage **configuration items (HW/SW/combos)** · ensure system **performance/function/integrity** · document changes · inform everyone (transparency) · enable **concurrent/distributed engineering** · enable **roll-backs** · create **reproducible** configurations.
- **5-step process (O-I-C-S-V):** ① **Organization** (rules, tools, naming, training) ② **Identification** (units, product structure, base line, granularity) ③ **Control** ← **change management happens HERE** ④ **Status accounting** (document config + item properties/versions) ⑤ **Verification & audit** (test the requirements).
- **Standards:** EIA-649C · Automotive SPICE (SUP) · IEEE 828-2012 · ISO 10007 · NIST. Needed to pass **ISO 26262 ch.8** (vehicle FuSi) & **ISO 20000** (IT service mgmt).



LEGO combo = config; one brick = change. Process = **O-I-C-S-V** (control = where change lives).

## 11 Variant management (paint-swatch versions)

#### STORY

Ada lines up the same Volt in several paint-and-trim versions. Underneath, the same base product → parts bought in bulk → **economy of scale**. Customers see different looks (HMI, colour, shape) and **features** (personal vs enterprise). More variants = more money but **more effort**.

#### WRITE IN EXAM

- **Basics:** business model defines the variants. Tasks: transparency · optimize (create/delete) · link with **LCM**, impact **config-CM**. **Target:** as many as needed for revenue, as few as possible for complexity → standard interfaces & tool kits.
- **Extra effort by area:** **Development** (what must the base change?) · **Testing** (which tests can be skipped safely?) · **Sales** (don't let variants “negative” each other; build IT-configurators) · **General** (banned variants? cannibalization?).
- **Increase variants** → customization · new **markets** (US vs EU) · new **clienteles** (teens→silver agers) · new **use cases** (craft→medical). **Decrease** → cut cost via **synergies** (one plug for all).
- **Economy & ROI:** new variant = **low extra cost** (base covers it), **fewer customers**, but **ROI usually higher** than the base.



Same base under the shell = **economy of scale**. Target = **as needed / as few as possible** with tool kits.

## 12 Loose ends + Summary



### WRITE IN EXAM

- **Software — version control (code):** Azure DevOps · BitKeeper · BitBucket · **Git** · IBM Rational Synergy · RCS · Subversion.
- **Software — product portfolio:** cplace · ClickUp.
- **Summary:** in an ideal project **no CM is needed**; in reality CM gives **traceability · releasable/conform product · cost control · fewer delays/issues · high quality**. Spirit: “as few changes as possible, but as many as needed.”



**Closing message:** “While change management is important, **don't be afraid of changes!**”

YOUR PROGRESS

**0 / 12 topics learned**

---

# Flashcards

Read the question, say the answer *out loud*, then flip. Cover the deck and run it twice — that's how it sticks.

## All flashcards (Q/A)

### 1. Two module competences for Change Management?

Understand the necessity of CM (level 1); apply the CM process to projects (level 3).

### 2. Which System Engineering step is Change Management?

Step 4 — after Risk, before Ideation; sits between PRD freeze and Re-lease.

### 3. What is engineering change management (CM)?

The process of changing a product or parts of it (Ger. Änderungsmanagement, ÄM).

### 4. The 'other' CM we DON'T mean here?

Organizational change management (Veränderungsmanagement) — processes, behaviours, strategy, structures.

### 5. Content & target of CM?

Content: align, track, follow up requirement/request changes. Target: as few changes as possible but as many as needed.

### 6. When is a change NOT change management?

When intended by the product itself (e.g. a chameleon). A change of the product DESIGN is CM.

### 7. VM / LCM / CM / CM by object?

VM = products/variants; LCM = new generations; Config-CM = HW+SW configurations; Change-CM = components. Count grows top→down.

### 8. Example object for each cousin?

VM: sedan/convertible. LCM: E90→F30→G20. Config-CM: ECU SW 3.7.2 + d-sample PCB. Change-CM: the PCB.

### 9. Which two disciplines both abbreviate 'CM'?

Configuration management and Change management.



**10. 4 factors that increase a change's effort?**

System complexity, late point of detection (Rule of Ten), overlapping changes, variants.

**11. The 4 phases of WHEN to do CM?**

Until SOP (most changes); launch management (critical, just before SOP); SOP→EOP (trouble-shooting & cost-down); after EOP (field issues, next product).

**12. WHERE to document changes + key rule?**

Two IT locations: project-management level + where the original object was created (PDM). Both MUST be linked, or data branches.

**13. For whom is CM, and supplier rule?**

All internal AND bought/supplied products; track suppliers, nominate ISO 9001-certified ones, put the reporting duty in the contract.

**14. Why do change management?**

Design-for-upgrade, traceability (versioning), releasable/conform product, cost control, fewer delays/issues, high quality.

**15. Level vs amount of changes?**

The higher the level of change, the fewer such changes occur.

**16. The 3 TYPES of change?**

Scope-Enlargement (add new), Scope-Reduction (cancel), Scope-Change (change existing content).

**17. Evolutionary vs Revolutionary change?**

Evolution = gradual improvement (better phone). Revolution/disruption = radical rethink (smartphone vs telephone).

**18. Motivation for version control?**

Traceability over the whole life cycle: change frequency & evolution visible; roll-back possible.

**19. Key principle when changing an object?**

Never overwrite — create a NEW version; version number is an attribute (auto-counter).

**20. How is an object uniquely identified?**

By ID/name + version number together (e.g. #073 alone isn't enough — v1.0 or v1.1?).

### **21. Version-numbering schemes?**

Single-level numeric, single-level alphabetical, and two-stage (e.g. 1.1,1.2,2.0).

### **22. MAJOR.MINOR.MICRO from 3.9.4?**

4.9.4 = major (re-design), 3.10.4 = minor (feature, not 4.x!), 3.9.5 = micro (patch).

### **23. Version-control PROCESS?**

Like new requirements: change to pool → acceptable? → no: decline/modify → yes: release new version.

### **24. What to log in version control?**

Object, status, version, what changed, why, for whom, who changed, when, who approved, when approved.

### **25. Document control per ISO 9001?**

Available where/when needed; protected; managed for distribution/access/discovery/use; stored with preserved readability; change-monitored; external docs marked.

### **26. Where to keep a machine's safety manual?**

Directly at the machine, not only on a digital drive.

### **27. What can be the impact of a change?**

Complexity ↑ (quality issues, unhappy customers), time delay, worse cost/benefit, breaks the PRD, worst case the project is stopped.

### **28. The 'inversed Rule of Ten'?**

The later the ROOT CAUSE (not detection) in the design process, the cheaper & easier to change.

### **29. Inversed Rule of Ten — cheap→dear order?**

Update design → update assembly process → change concept → redesign test → change whole system.

### **30. Causes of a design & concept change?**

Failed tests; realization issues (HW won't work/too costly → SW); needing a re-design or new concept.

### **31. Ideal vs reality of the PRD freeze?**

Ideal: hard freeze, no changes after. Reality: partly soft, so changes are needed.

**32. 'Have-to' requirement changes?**

Mandatory before launch: a changing law, or a failed verification → relax an over-engineered requirement.

**33. 'Can-be' requirement changes?**

Your call: new applications, company strategy, customer expectation (B2B cost!), tool kits, cultural/political.

**34. How to handle a requirement change?**

Re-open requirement process → impact analysis → create a CR → CCB decides & records → prioritize → plan.

**35. Current valid configuration equals?**

Base line (reference) + all CRs.

**36. What manages repeated product generations?**

Product lifecycle management (PLM/LCM).

**37. Positive vs negative reasons to repeat the cycle?**

Positive: new requirements, intrinsic advancement, state-of-art. Negative: competing products, quality issues, bad reviews, cost reduction.

**38. Horizontal vs vertical networking?**

Horizontal = across generations (forward-only). Vertical = across products, bidirectional, by 'technical proximity'.

**39. What 4 things can a life-cycle branch do?**

Continue, merge, split, or die (e.g. Linux distribution timeline).

**40. What affects life-cycle length?**

Industry (auto ~7 yrs vs phone ~1 yr), new technology, and the starting point; trend = shorter cycles.

**41. New / wide / close derivation?**

New = nothing reused; wide = basic features inherited; close = most carried over with minor tweaks.

**42. The 6 life-cycle phases?**

0 Investment, 1 Introduction, 2 Growth, 3 Maturity, 4 Saturation, 5 Degradation.

**43. Phase 0 & 1 characteristics?**

0 Investment: high cost & uncertainty, as short as possible. 1 Introduction: slow sales, no profit, ends at break-even.

**44. Phase 3 & 5 characteristics?**

3 Maturity: most profitable, as long as possible. 5 Degradation: negative profit, take it off the market.

**45. Aim of LCM?**

Coordinate product cycles to avoid fluctuations — continuous production, constant workload, consistent revenue (no 'hole').

**46. LCM measures to lengthen a cycle?**

Cost measures (remove unneeded functions/parts, cut over-spec) and product enrichment (new features).

**47. What is the BCG matrix + its limit?**

BCG plots market growth × relative market share for product decisions; records actuals only — NOT a forecast.

**48. BCG: Star — properties & decision?**

High growth, high share, best product → invest, or skim without losing share.

**49. BCG: Cash Cow — properties & decision?**

Low growth, high share → milk/skim, no discounts.

**50. BCG: Question Mark — properties & decision?**

High-growth, low share, new product → invest from other sources or discontinue.

**51. BCG: Poor Dog — properties & decision?**

Low growth, low share → terminate (latest when contribution margin/DB < 0).

**52. Typical BCG order in a life cycle?**

Question Mark → Star → Cash Cow → Poor Dog.

**53. Relative market share formula?**

Company market share ÷ competitor market share (>1 = you lead).

**54. Configuration vs change management?**

Config = the combination of changes & resulting properties; change = the single change/measure.

**55. Targets of configuration management?**

Handle variant complexity, manage configuration items, ensure system integrity, document, inform, enable concurrent engineering & roll-backs, reproducible configs.

**56. The 5 steps of the CM process?**

Organization, identification, control, status accounting, verification & audit (O-I-C-S-V).

**57. In which CM step does change happen?**

Step 3 — configuration control.

**58. Standards describing configuration management?**

EIA-649C, Automotive SPICE (SUP), IEEE 828-2012, ISO 10007, NIST; needed for ISO 26262 ch.8 & ISO 20000.

**59. Variant management + its target?**

Business model defines variants. Target: as many as needed for revenue, as few as possible for complexity — use standard interfaces & tool kits.

**60. How does a shared base product help variants?**

Same base → components sourced more often → cheaper per piece (economy of scale); differentiate by look & features.

**61. Where do variants cause extra effort?**

Development, testing, sales, and general (banned variants? cannibalization?).

**62. Increase vs decrease variants?**

Increase: customization, new markets/clientele/use cases. Decrease: cut cost via synergies (one plug for all).

**63. Economy & ROI of a variant?**

Low extra cost (base covers main costs), fewer customers, but ROI typically higher than the base.

**64. Variant decision: base+ deriv- sum+ ?**

Case 3: implement only if the variant's economy of scale is why the base is positive.

**65. Variant decision: base+ deriv- sum- ?**

Case 4: don't implement; check the base is still BC-positive.

**66. Software tools for code version control?**

Azure DevOps, BitKeeper, BitBucket, Git, IBM Rational Synergy, RCS, Subversion.

**67. Software tools for product portfolio?**

cplace and ClickUp.

**68. Summary: when is no CM needed, why needed in reality?**

Ideally never. Reality: traceability, releasable/conform product, cost control, fewer delays/issues, high quality.

**69. Closing message of the chapter?**

While change management is important, don't be afraid of changes.

---

## 3 Practice Questions + model answers

Try to answer on paper before clicking **Reveal**. The model answers show the exact points an examiner rewards.

### Q1 Why is change management needed even after the PRD freeze? Describe when (the phases) and where it is done, and how a requirement change is processed.

TYPE: STRUCTURED / THEORY · ~10 MARKS

REVEAL MODEL ANSWER

#### ✓ MODEL ANSWER

The ideal hard PRD freeze is in reality soft: laws change, tests fail, suppliers swap parts. The goal is “as few changes as possible, but as many as needed” (stable development, yet a sellable product).

- **When** — 4 phases: until SOP (most changes), **launch management** (just before SOP, critical), SOP→EOP (trouble-shooting + cost-down), **after EOP** (record field issues to improve the next product).
- **Where**: documented at two linked IT locations — the **project-management level** (whole change) and **where the original object was made** (e.g. a PDM system). Both must be **linked** or the data branches.
- **Requirement change process**: re-open the requirement process → **impact analysis** (resources + risks) → create a **CR** → the **CCB** (or project lead) decides & records → **prioritize** (cost/benefit) → **plan**. The valid configuration = **base line** + **all CRs**; then update, document (versioning), implement, track verification, release.

Changes are either “**have-to**” (new law, failed verification) or “**can-be**” (new applications, strategy, customer expectation, tool kits, cultural/political).

**Q2**

**Distinguish change management, configuration management, variant management and life-cycle management. Then list the 5 steps of the configuration-management process and say where change actually happens.**

TYPE: THEORY / DEFINITIONS · ~12 MARKS

REVEAL MODEL ANSWER

✓ MODEL ANSWER

They differ by the **object** they manage (object count grows down the list):

- **Variant management (VM):** products/variants (sedan, convertible).
- **Life-cycle management (LCM):** new generations (E90→F30→G20); coordinates all cycles to avoid “holes”.
- **Configuration management (CM):** HW+SW configurations and their resulting properties.
- **Change management (CM):** a single component / single change & measure.

Configuration vs change: **change** focuses on the **single change**; **configuration** on the **interaction of changes** and resulting properties.

CM 5-step process (O-I-C-S-V): ① CM organization, ② configuration identification (units, product structure, base line, granularity), ③ **configuration control — change management happens here**, ④ status accounting, ⑤ verification & audit (test the requirements). Standards: EIA-649C, Automotive SPICE (SUP), IEEE 828-2012, ISO 10007 — relevant for ISO 26262 ch.8 and ISO 20000.



**Q3**

**(a) Sketch the product life-cycle curve and name its phases. (b) Explain the BCG matrix and its typical order. (c) A new variant has a positive base BC, a negative derivation BC, but a positive sum — what do you do, and why?**

TYPE: APPLICATION · ~8 MARKS

REVEAL MODEL ANSWER

✓ MODEL ANSWER

(a) Phases over time (sales/revenue): **0 Investment** (high cost, short) → **1 Introduction** (slow, to break-even) → **2 Growth** (steep) → **3 Maturity** (most profitable, long) → **4 Saturation** (sales fall) → **5 Degradation** (negative profit, take it off the market). Key markers: market launch, break-even, EOS.

(b) The **BCG matrix** (Boston Consulting Group) plots products on **market growth × relative market share** to record the actual portfolio (not a forecast): **Stars** (invest/skim), **Cash Cows** (milk), **Question Marks** (invest from elsewhere or drop), **Poor Dogs** (terminate). Typical order: **Question Mark** → **Star** → **Cash Cow** → **Poor Dog**; question-marks create future stars, cash cows finance them.

(c) This is **case 3**: implement the variant **only** if the variant's additional **economy of scale** is the reason the base variant's business case is positive — i.e. removing the derivation would raise the base's costs and worsen the overall sum. Otherwise don't.

— LAST 5 MINUTES BEFORE THE EXAM

# One-screen cheat-card

|                     |                                             |                                  |
|---------------------|---------------------------------------------|----------------------------------|
| Spirit              | as few as possible, as many as needed       | keep stable, stay sellable       |
| CM vs not           | re-paint blueprint (yes) vs chameleon (no)  | design change = CM               |
| 4 cousins           | VM ▶ LCM ▶ Config-CM ▶ Change-CM            | Russian dolls, counts grow       |
| Two "CM"s           | Configuration vs Change                     | combo vs single brick            |
| Effort grows with   | complexity · detection · overlap · variants | Tangled·Late·Crowded·Many        |
| When (4)            | preSOP · launch · SOP-EOP · postEOP         | Build·Launch·Fix·Learn           |
| Where (2)           | project level + PDM, LINKED                 | two drawers, one chain           |
| For whom            | internal + suppliers (ISO 9001 + contract)  | watch the supplier               |
| Type (3)            | enlarge · reduce · change                   | inflate · pop · reshape          |
| Size (2)            | evolution vs disruption                     | tortoise vs rocket               |
| Versioning          | MAJOR.MINOR.MICRO, never overwrite          | 3.9.4 → 3.10.4 (not 4.x)         |
| Version content     | what·why·who·when·approve                   | detective's W-form               |
| Doc control         | available·protected·readable·marked         | manual at the machine (ISO 9001) |
| Inversed Rule of 10 | later root cause = cheaper                  | snowball rolls UP                |
| Req change          | have-to vs can-be → CR→CCB                  | cracked frozen blueprint         |
| Life-cycle phases   | 0 invest ... 5 degrade                      | roller-coaster                   |
| BCG                 | Star · Cash Cow · Question · Dog            | farmyard; ? → ★ → 🐮 → 🐶          |
| Config process      | O-I-C-S-V                                   | change lives in Control          |
| Variant decision    | read the SUM column                         | + = go, - = stop (case 3 trap)   |

Change Management · CPS MMC-01a · study build · keep Ada & her ever-changing car in your head



Downloads: [PDF](#) · [Anki flashcards](#) · [memorize guide](#) · [notes](#)